



Setting Up Full Multigrid

Louis Moresi¹ 

¹Australian National University

Keywords Underworld Code, Tricks of the Trade, development

Most of the time in a geodynamics model goes into solving the Stokes equations, and most of that goes into the velocity block. How that block is preconditioned decides how long a model takes to run, and it is one of the few settings where the right choice is worth an order of magnitude.

1. WHAT MULTIGRID DOES

An iterative solver reduces error unevenly. Simple relaxation methods — the smoothers — are good at removing error that varies rapidly from cell to cell, and poor at removing error that varies smoothly across the whole domain. The smooth part is the expensive part: on a fine mesh it takes many sweeps to move information from one side of the model to the other.

Multigrid removes that part somewhere else. Error that is smooth on a fine mesh is *not* smooth on a mesh twice as coarse, where it spans half as many cells, so a few sweeps on the coarse mesh remove what the fine mesh could not. The method is a recursion: smooth on the fine mesh, transfer what is left to a coarser one, solve there (by recursing again), transfer the correction back, smooth again. Each level handles the band of error it can see cheaply, and the work per unknown stops growing as the model gets bigger.

Two things have to be supplied: the coarse levels, and the operators that move between them.

2. TWO WAYS TO BUILD THE COARSE LEVELS

Algebraic multigrid builds them from the matrix. It inspects the operator's connectivity, groups unknowns that are strongly coupled, and constructs coarse levels and transfer operators from that grouping alone. It needs nothing from the mesh, which is why it works everywhere and is the sensible default. PETSc's implementation is GAMG, and it is what Underworld uses when it has nothing better.

Geometric multigrid uses actual coarser meshes. If the fine mesh was built by refining a coarse one, those coarser meshes already exist, and the transfers follow from the refinement relation: each fine node either coincides with a coarse node or lies inside a known coarse cell. In PETSc this is `pc_type=mg`, and run as a *full* multigrid cycle — starting on the coarsest level and working up, rather than starting fine — it is what we call FMG.

The difference matters when the operator's connectivity is a poor guide to the geometry, which is exactly what a strong viscosity contrast produces. The algebraic method sees a matrix whose couplings are dominated by the stiff region and groups unknowns accordingly; the geometric method does not care, because it was told the grids.

3. MAKING A MESH THAT HAS A HIERARCHY

A mesh only carries a hierarchy if it was built with one. That is the `refinement` argument:

```
mesh = uw.meshing.UnstructuredSimplexBox(cellSize=0.05, refinement=2)
len(mesh.dm_hierarchy)      # 3: the base and two refinements
```

The mesh you get back is the finest level, and the coarser ones are stored within it as PETSc `DMPlex` objects in `mesh.dm_hierarchy`. They are what the preconditioner uses. They are not Underworld meshes: there is no `Mesh` object for a coarse level, and building one takes work, so the hierarchy is something the solver reads rather than something you interact with.

Build the same mesh at `cellSize=0.0125` with no refinement and you get about the same resolution with no hierarchy at all — and no geometric multigrid.

The solver picks it up on its own:

```
stokes = uw.systems.Stokes(mesh)
stokes.preconditioner      # "auto" by default
stokes.solve()
```

"auto" uses geometric multigrid when `len(mesh.dm_hierarchy) > 1` and GAMG when it does not. You can also ask directly, with "fmg" or "gamg". Two behaviours are deliberate: "auto" only ever adds geometric multigrid to an untouched configuration, so it will not overwrite preconditioner options you have set yourself; and where a request cannot be honoured, the reason is recorded in `stokes.pc_fallbacks` instead of the solve quietly running on something else. `stokes.preconditioner_settings` reports what was actually applied.

4. CURVED BOUNDARIES HAVE TO BE TOLD THEY ARE CURVED

Refining a mesh puts new nodes at the midpoints of existing edges. On a straight boundary that is fine. On a curved one it is not: the coarse mesh approximates a circle by a polygon, and the midpoint of a chord lies inside the circle, not on it. Refine naively and every level keeps the coarsest level's polygon, so the geometry never improves no matter how fine the mesh becomes.

Underworld's curved meshes carry a refinement callback that snaps boundary-labelled nodes back onto the true surface after each refinement. For the annulus it is a few lines — take the nodes labelled `Upper` and `Lower` and rescale each to the correct radius:

```
coords[upper] *= radiusOuter / R[upper]
coords[lower] *= radiusInner / R[lower]
```

This happens automatically for the built-in meshes, so in normal use there is nothing to do. It matters if you build a mesh of your own with curved boundaries: without a callback of this kind the hierarchy is geometrically wrong, and the error does not go away with refinement.

5. WHAT THE COARSE MESH LEAVES BEHIND

Because the fine mesh is made by subdividing the coarse one, the coarse mesh's own triangulation is still visible in it. Its edges survive as continuous lines across the fine mesh, and its vertices remain places where an unusual number of elements meet. The pattern is a record of how the mesh was made rather than of the problem being solved.

`mesh.relax()` moves the nodes to improve element shapes while keeping the resolution and the topology, which loosens that imprint.

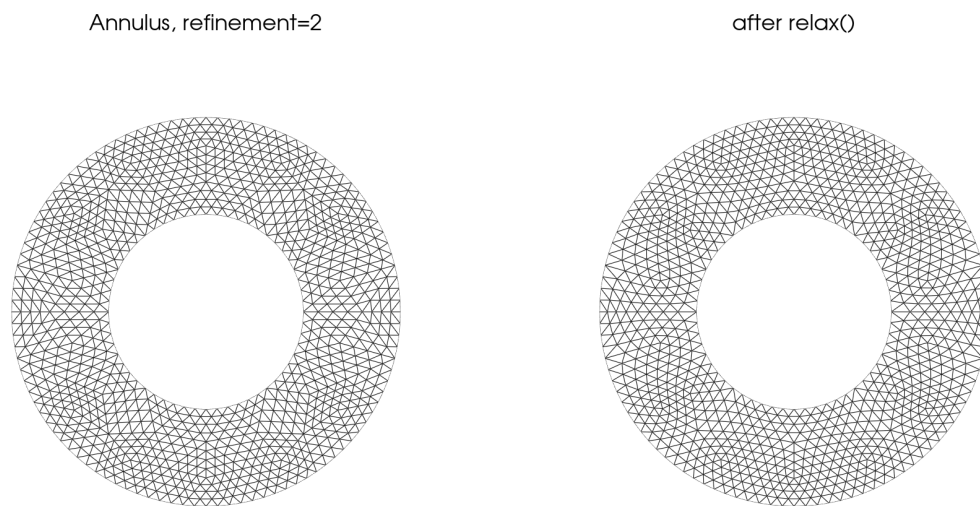


Figure 1: A twice-refined annulus, before and after `relax()`. The seams on the left are the coarse mesh's own edges, still visible after two refinements. Median element quality goes from 0.940 to 0.972 and the tenth percentile from 0.839 to 0.927; the worst single cell goes the other way, 0.827 to 0.763, because the mover minimises a global energy and will trade a few cells for many. Every node that began on a bounding circle is still exactly on it: relaxation moves interior coordinates only.

The effect is clearer on the annulus than on a box, whose base mesh is already good — there the same operation moves median quality only from 0.976 to 0.994. Because relaxation moves coordinates and not topology, the hierarchy survives it, which is the subject of a [companion note](#).

6. WHEN THE CHOICE MATTERS

The test is SolKz, a standard Stokes benchmark on the unit box: viscosity varying exponentially with depth, $\eta = e^{2Bz}$, forced by $\mathbf{f} = (0, \sin(m\pi z) \cos(n\pi x))$ with $n = 3, m = 2$, free slip on all four walls, Taylor–Hood P_2 – P_1 elements. The viscosity contrast across the box is e^{2B} . SolKz has a closed-form solution, but nothing here uses it: these are timings, not accuracy measurements.

Effort is reported per unknown and relative to the cheapest FMG run, which makes it a ratio rather than a time. A ratio does not depend on the machine it was measured on, so the table means the same thing wherever it is read.

First, cost against viscosity contrast at a fixed resolution of 11 727 unknowns:

preconditioner	viscosity contrast	velocity iterations	relative effort per unknown
FMG	10^0	48	1.00
FMG	10^2	66	1.12
FMG	10^4	84	1.25
FMG	10^6	198	2.67
GAMG	10^0	4 620	3.45
GAMG	10^2	5 913	4.27
GAMG	10^4	6 647	4.75
GAMG	10^6	14 388	10.02

Both work at every contrast. FMG costs about three times more at 10^6 than at constant viscosity; GAMG costs ten times more than the cheapest FMG run at constant viscosity, and ten times *that* by 10^6 .

Second, cost against problem size at constant viscosity:

preconditioner	unknowns	velocity iterations	relative effort per unknown
FMG	2 947	48	1.00
FMG	11 727	48	1.23
FMG	46 783	48	1.50
FMG	186 879	48	1.82
GAMG	2 947	1 726	2.00
GAMG	11 727	4 620	4.33
GAMG	46 783	14 574	13.04

This is what multigrid exists for. Across a 64-fold growth in the problem, the effort FMG spends per unknown does not quite double. Fitted against problem size the solve time goes as $N^{1.16}$, against $N^{1.69}$ for GAMG.

The iteration column says where the difference comes from. FMG takes **exactly 48 velocity iterations at every size** — the count is independent of the mesh, which is the property multigrid is built to have. Everything above linear in its timing is the cost of an iteration rising as the hierarchy deepens, not more iterations being needed. GAMG’s iteration count grows as roughly $N^{0.7} - N^{0.8}$, and that exponent is itself increasing.

Neither is $N \log N$: that would predict growth factors of 4.67, 4.58 and 4.51 across the three refinement steps, against the 5.21, 4.92 and 4.83 measured.

Two things this comparison does not settle

The solver tolerance is held fixed as the mesh refines. That is the usual way to show a multigrid scaling and it is a choice: a finer mesh has a smaller discretisation error, so an argument can be made for tightening the solver tolerance alongside it, which would make the effort per unknown grow. The table answers “what does it cost to solve this system”, not “what does it cost to reach the accuracy the mesh can support”.

The GAMG column is GAMG *as Underworld configures it*. Algebraic multigrid on an elasticity-like operator wants the rigid-body near-null space, and Underworld does not currently supply it for the Stokes velocity block, so these numbers are the default rather than the best algebraic multigrid can do.

SolKz is also a gentle test, because its viscosity varies smoothly everywhere. Replace the smooth profile with a *localised* viscous layer and GAMG stops converging at all above a contrast of about 10^2 , while FMG is barely affected. Smoothness is what algebraic coarsening reads, and structure that is concentrated rather than distributed is what takes it away.

7. USING IT

```
# Build the hierarchy at mesh construction: base + two refinements.
mesh = uw.meshing.UnstructuredSimplexBox(cellSize=0.05, refinement=2)

stokes = uw.systems.Stokes(mesh)
stokes.preconditioner = "auto"      # FMG when a hierarchy exists
stokes.solve()

print(len(mesh.dm_hierarchy))      # how many levels there are
print(stokes.preconditioner_settings) # what was applied
print(stokes.pc_fallbacks)         # anything declined, and why
```

If a solve is slow and the mesh has no hierarchy, that is the first thing to change: rebuild the mesh with `refinement` rather than at a fine `cellSize`, and the same resolution arrives with the coarse levels attached.